Automated Poultry Farm for Advanced Chicken Farming

Prepared By:
Ramon Francisco
Edilyn Mallo
Jexel Talaba

DESCRIPTION

This system automates poultry farming by managing water and food supply while tracking chick growth. It integrates sensors and a microcontroller to enhance efficiency and reduce manual work.

1. **Water Dispensing**

-A Water Level Sensor monitors the tank's water level. When it drops below a threshold, the Relay Module activates the Water Pump to refill it. Once the desired level is reached, the system stops the pump, ensuring a consistent water supply.

DESCRIPTION

2. **Food Dispensing**
The HW 111 Real Time Clock detects food time and signals the Servo Motor to open the dispenser, ensuring an automated feeding process. This prevents food shortages and maintains a steady diet for the poultry.

3. **Chick Weight & Identification**
Each chick has an RFID Sticker scanned by the RC522 RFID Sensor for identification. The HX711 Weight Sensor measures the chick's weight, and the SIM800A GSM Module sends updates via SMS to track growth and health.

MAIN COMPONENTS

Microcontroller

- ESP32 Microcontroller – Controls all system operations, processes sensor inputs, and manages automation.



ESP32 Microcontroller

Inputs (Sensors)

- Water Level Sensor – Detects water levels in the tank
- HX711 Weight Sensor + Load Cell – Measures the chick weight.
- RC522 RFID Sensor – Reads RFID tags for chick identification.
- HW-111 Real Time Clock - Detects time for food distribution



Water Level Sensor



HX711 Weight Sensor + Load Cell



HW-111



RC522 RFID module

MAIN COMPONENTS

Outputs (Actuators)

- **Water Pump** – Activates when the water level is low.
- **Relay Module** – Controls the water pump and other high-power components.
- **Servo Motor** – Opens the food dispenser when triggered.
- **SIM800A GSM Module** – Sends SMS notifications about chick weight and health.



Water Pump



Servo Motor

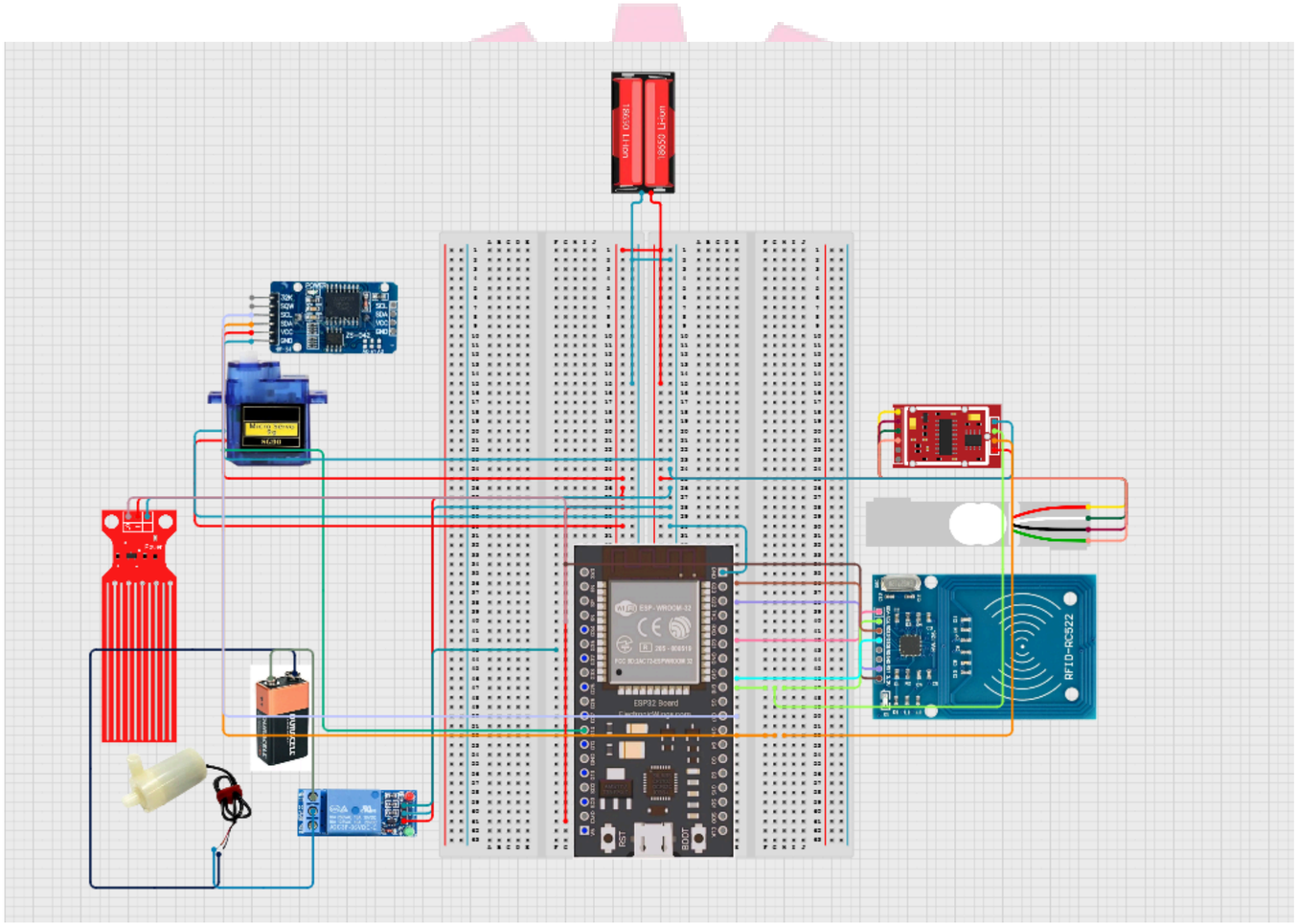


Relay Module



SIM800A GSM Module

SCHEMATIC DIAGRAM



CODES

```
#include <ESP32Servo.h>
#include <WiFi.h>
#include <WebServer.h>
#include "time.h"
#include <SPI.h>
#include <MFRC522.h>
#include <HardwareSerial.h>
#include <EEPROM.h>
#include <ArduinoJson.h>
#include <HX711.h>

// WiFi Configuration - Dual mode (STA + AP)
const char* wifiSSID = "4G";
const char* wifiPassword = "09472713552Fr@ncisco";
const char* apSSID = "ChickenFeeder_AP";
const char* apPassword = "12345678";

// IP Configuration for AP
IPAddress localIP(192, 168, 4, 1);
IPAddress gateway(192, 168, 4, 1);
IPAddress subnet(255, 255, 255, 0);

WebServer server(80);

// NTP Settings
const char* ntpServer = "time.windows.com";
const long gmtOffset_sec = 28800;  // GMT+8
const int daylightOffset_sec = 0;

// Pin Definitions
#define SERVO_PIN 14
#define WATER_SENSOR_PIN 34
#define RELAY_PIN 32
#define RST_PIN 22
#define SS_PIN 21
#define SIM800L_RX 16
#define SIM800L_TX 17
#define SCK_PIN 18
#define MOSI_PIN 23
#define MISO_PIN 19
#define HX711_DOUT 5
#define HX711_SCK 4

// Feeding schedule
const int feedHours[] = {6, 12, 15, 20};
const int feedMinutes[] = {0, 0, 0, 28};
const int numFeeds = 4;

// Global Variables
Servo myservo;
MFRC522 rfid(SS_PIN, RST_PIN);
HardwareSerial sim800l(1);
HX711 scale;  // HX711 instance
int lastFeedHour = -1;
int lastFeedMinute = -1;
bool lastWaterState = false;
String lastRFIDTag = "N/A";
const String PHONE_NUMBER = "+639672688658";
unsigned long lastSerialPrintTime = 0;
const unsigned long serialPrintInterval = 5000;

// EEPROM Settings
#define EEPROM_SIZE 4096
#define MAX_TAGS 50
#define TAG_DATA_SIZE 64  // Enough space for tag ID, name, and weight data

// HX711 Scale Settings
float calibration_factor = -7050.0;  // Adjust this value for your scale
float weight = 0;
bool scaleReady = false;

// Tag Management
struct TagRecord {
 String tagId;
 String tagName;
 float lastWeight;
 String lastScanTime;
};

TagRecord tagDatabase[MAX_TAGS];
int tagCount = 0;

// Modern HTML UI with Tag Management
const char* htmlPage = R"rawliteral(
<!DOCTYPE HTML>
<html>
<head>
  <title>Chicken Feeder Control</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
   :root {
    --primary: #3498db;
    --success: #2ecc71;
    --danger: #e74c3c;
    --dark: #2c3e50;
    --light: #ecf0f1;
   }
   body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: #f5f7fa;
    margin: 0;
    padding: 20px;
    color: var(--dark);
   }
   .container {
    max-width: 1000px;
    margin: 0 auto;
   }
   .header {
    background: white;
    padding: 20px;
    border-radius: 10px;
    margin-bottom: 20px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.05);
    text-align: center;
   }
```

CODES

```
h1 {
 color: var(--primary);
 margin-bottom: 10px;
}
.status-bar {
 display: flex;
 justify-content: space-between;
 font-size: 14px;
}
.badge {
 padding: 3px 8px;
 border-radius: 10px;
 font-size: 12px;
 font-weight: 600;
}
.online {
 background: rgba(46, 204, 113, 0.2);
 color: var(--success);
}
.offline {
 background: rgba(231, 76, 60, 0.2);
 color: var(--danger);
}
.card {
 background: white;
 border-radius: 10px;
 padding: 20px;
 margin-bottom: 20px;
 box-shadow: 0 2px 5px rgba(0,0,0,0.05);
}
.card-title {
 font-size: 18px;
 margin-bottom: 15px;
 color: var(--primary);
 border-bottom: 1px solid #eee;
 padding-bottom: 10px;
}
.sensor-row {
 display: flex;
 justify-content: space-between;
 margin-bottom: 15px;
 align-items: center;
}
.sensor-value {
 font-size: 20px;
 font-weight: 600;
}
.water-low {
 color: var(--danger);
}
.water-ok {
 color: var(--success);
}
.btn {
 background: var(--primary);
 color: white;
 border: none;
 padding: 12px 20px;
 border-radius: 6px;
 font-size: 16px;
 cursor: pointer;
 margin-right: 10px;
 transition: all 0.2s;
}
.btn:hover {
 opacity: 0.9;
 transform: translateY(-1px);
}
.btn:active {
 transform: translateY(0);
}
.btn-danger {
 background: var(--danger);
}
.btn-success {
 background: var(--success);
}
.btn-warning {
 background: #f39c12;
}
.rfid-tag {
 background: #f8f9fa;
 padding: 8px 12px;
 border-radius: 6px;
 font-family: monospace;
}
.schedule-item {
 display: flex;
 justify-content: space-between;
 padding: 8px 0;
 border-bottom: 1px solid #eee;
}
.feeding-now {
 color: var(--success);
 font-weight: 600;
}
table {
 width: 100%;
 border-collapse: collapse;
 margin-top: 15px;
}
th, td {
 padding: 12px 15px;
 text-align: left;
 border-bottom: 1px solid #ddd;
}
th {
 background-color: #f2f2f2;
 font-weight: 600;
}
tr:hover {
 background-color: #f5f5f5;
}
.modal {
 display: none;
 position: fixed;
 z-index: 1;
 left: 0;
 top: 0;
 width: 100%;
 height: 100%;
 overflow: auto;
 background-color: rgba(0,0,0,0.4);
}
.modal-content {
 background-color: #fefefe;
 margin: 15% auto;
 padding: 20px;
 border-radius: 8px;
 width: 80%;
 max-width: 500px;
}
.close {
 color: #aaa;
 float: right;
 font-size: 28px;
 font-weight: bold;
 cursor: pointer;
}
.close:hover {
 color: black;
}
.form-group {
 margin-bottom: 15px;
}
.form-group label {
 display: block;
 margin-bottom: 5px;
 font-weight: 600;
}
.form-group input {
 width: 100%;
 padding: 8px;
 border: 1px solid #ddd;
 border-radius: 4px;
 box-sizing: border-box;
}
@media (max-width: 600px) {
 .btn {
  width: 100%;
  margin-bottom: 10px;
 }
 .modal-content {
  width: 95%;
 }
}
</style>
</head>
<body>
 <div class="container">
  <div class="header">
   <h1>Chicken Feeder Control</h1>
   <div class="status-bar">
    <span>WiFi: <span id="wifi-status" class="badge offline">Disconnected</span></span>
```

CODES

```
    <span>RFID: <span id="rfid-status" class="badge offline">Offline</span></span>
    <span>Scale: <span id="scale-status" class="badge offline">Offline</span></span>
    <span id="current-time">--:--:--</span>
   </div>
  </div>

  <div class="card">
   <h2 class="card-title">Water System</h2>
   <div class="sensor-row">
    <span>Water Level:</span>
    <span id="water-level" class="sensor-value water-ok">--</span>
   </div>
   <div style="margin-top: 15px;">
    <button id="pump-btn" class="btn btn-success" onclick="togglePump()">Turn Pump ON</button>
   </div>
  </div>

  <div class="card">
   <h2 class="card-title">Feeding System</h2>
   <div style="margin-bottom: 15px;">
    <button class="btn" onclick="manualFeed()">Manual Feed</button>
    <button class="btn btn-warning" onclick="weighChicken()">Weigh Chicken</button>
   </div>
   <div class="sensor-row">
    <span>Current Weight:</span>
    <span id="current-weight" class="sensor-value">-- kg</span>
   </div>
   <div class="sensor-row">
    <span>Last RFID:</span>
    <span id="rfid-tag" class="rfid-tag">--</span>
   </div>
  </div>

  <div class="card">
   <h2 class="card-title">Tag Management</h2>
   <button class="btn" onclick="openTagModal()">Add/Edit Tag</button>
   <div id="tag-table-container">
    <table id="tag-table">
     <thead>
      <tr>
       <th>Tag ID</th>
       <th>Name</th>
       <th>Last Weight (kg)</th>
       <th>Last Scan</th>
       <th>Actions</th>
      </tr>
     </thead>
     <tbody>
      <!-- Tag data will be inserted here -->
     </tbody>
    </table>
   </div>
  </div>

  <div class="card">
   <h2 class="card-title">Feeding Schedule</h2>
   <div id="schedule-display">
    <!-- Schedule will be inserted here -->
   </div>
  </div>
 </div>

 <!-- Tag Management Modal -->
 <div id="tagModal" class="modal">
  <div class="modal-content">
   <span class="close" onclick="closeTagModal()">&times;</span>
   <h2 id="modal-title">Add New Tag</h2>
   <div class="form-group">
    <label for="modal-tag-id">Tag ID:</label>
    <input type="text" id="modal-tag-id" readonly>
   </div>
   <div class="form-group">
    <label for="modal-tag-name">Tag Name:</label>
    <input type="text" id="modal-tag-name" placeholder="Enter tag name">
   </div>
   <div class="form-group">
    <label for="modal-tag-weight">Weight (kg):</label>
    <input type="number" id="modal-tag-weight" step="0.01" placeholder="Enter weight">
   </div>
   <button class="btn" onclick="saveTag()">Save Tag</button>
  </div>
 </div>

 <script>
  // Auto-refresh every 1.5 seconds
  const updateInterval = 1500;
```

CODES

```
// Update pump button
const pumpBtn = document.getElementById('pump-btn');
pumpBtn.textContent = data.pumpStatus ? "Turn Pump OFF" : "Turn Pump ON";
pumpBtn.className = data.pumpStatus ? 'btn btn-danger' : 'btn btn-success';

// Update current weight
document.getElementById('current-weight').textContent = data.currentWeight + " kg";

// Update RFID tag
document.getElementById('rfid-tag').textContent = data.rfidTag || "--";

// Update schedule
let scheduleHTML = '';
data.schedule.forEach(item => {
 scheduleHTML += `
  <div class="schedule-item">
   <span>${item.time}</span>
   <span class="${item.status.includes('now') ? 'feeding-now' : ''}">${item.status}</span>
  </div>`;
});
document.getElementById('schedule-display').innerHTML = scheduleHTML;

// Update tag table
let tagTableHTML = '';
data.tags.forEach(tag => {
 tagTableHTML += `
  <tr>
   <td>${tag.tagId}</td>
   <td>${tag.tagName}</td>
   <td>${tag.lastWeight}</td>
   <td>${tag.lastScanTime}</td>
   <td>
    <button class="btn" onclick="editTag('${tag.tagId}', '${tag.tagName}', ${tag.lastWeight})">Edit</button>
    <button class="btn btn-danger" onclick="deleteTag('${tag.tagId}')">Delete</button>
   </td>
  </tr>`;
 });
 document.querySelector('#tag-table tbody').innerHTML = tagTableHTML;
})
.catch(error => {
 console.error('Error:', error);
 // Retry sooner if error
 clearTimeout(updateTimer);
 updateTimer = setTimeout(fetchData, 500);
});
}

function manualFeed() {
 fetch('/manualfeed', { method: 'POST' })
  .then(() => fetchData()) // Refresh data
  .catch(err => console.error('Error:', err));
}

function weighChicken() {
 fetch('/weighchicken', { method: 'POST' })
  .then(() => fetchData()) // Refresh data
  .catch(err => console.error('Error:', err));
}

function togglePump() {
 fetch('/togglepump', { method: 'POST' })
  .then(() => fetchData()) // Refresh data
  .catch(err => console.error('Error:', err));
}

// Tag Management Functions
function openTagModal(tagId = '', tagName = '', weight = 0) {
 const modal = document.getElementById('tagModal');
 const modalTitle = document.getElementById('modal-title');
 const modalTagId = document.getElementById('modal-tag-id');
 const modalTagName = document.getElementById('modal-tag-name');
 const modalTagWeight = document.getElementById('modal-tag-weight');

 if (tagId) {
  modalTitle.textContent = 'Edit Tag';
  currentModalTagId = tagId;
 } else {
  modalTitle.textContent = 'Add New Tag';
  currentModalTagId = '';
 }

 modalTagId.value = tagId || 'Scan a tag first';
 modalTagName.value = tagName || '';
 modalTagWeight.value = weight || '';
 modal.style.display = 'block';
```

CODES

```
function closeTagModal() {
 document.getElementById('tagModal').style.display = 'none';
}

function editTag(tagId, tagName, weight) {
 openTagModal(tagId, tagName, weight);
}

function saveTag() {
 const tagName = document.getElementById('modal-tag-name').value;
 const weight = document.getElementById('modal-tag-weight').value;
 const tagId = currentModalTagId || document.getElementById('modal-tag-id').value;

 if (!tagName) {
  alert('Please enter a tag name');
  return;
 }

 fetch('/savetag', {
  method: 'POST',
  headers: {
   'Content-Type': 'application/json',
  },
  body: JSON.stringify({
   tagId: tagId,
   tagName: tagName,
   weight: parseFloat(weight) || 0
  })
 })
 .then(() => {
  closeTagModal();
  fetchData(); // Refresh data
 })
 .catch(err => console.error('Error:', err));
}

function deleteTag(tagId) {
 if (confirm('Are you sure you want to delete this tag?')) {
  fetch('/deletetag', {
   method: 'POST',
   headers: {
    'Content-Type': 'application/json',
   },
   body: JSON.stringify({
    tagId: tagId
   })
  })
  .then(() => fetchData()) // Refresh data
  .catch(err => console.error('Error:', err));
 }
}

// Close modal when clicking outside of it
window.onclick = function(event) {
 const modal = document.getElementById('tagModal');
 if (event.target == modal) {
  closeTagModal();
 }
}

 // Initial load
 fetchData();
 // Set up regular updates
 updateTimer = setInterval(fetchData, updateInterval);
 </script>
</body>
</html>
)rawliteral";

// Function to display connection info
void displayNetworkInfo() {
  Serial.println("\n📶 Network Details:");
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("- STA Mode (Client):");
    Serial.print("  SSID: ");
    Serial.println(WiFi.SSID());
    Serial.print("  IP: ");
    Serial.println(WiFi.localIP());
    Serial.print("  RSSI: ");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");
  } else {
    Serial.println("- STA Mode: Disconnected");
  }
  Serial.println("- AP Mode:");
  Serial.print("  SSID: ");
  Serial.println(apSSID);
  Serial.print("  IP: ");
  Serial.println(WiFi.softAPIP());
}

// AT Command Handling
String sendATCommand(String command, int timeout) {
  String response = "";
  sim800l.println(command);
  long startTime = millis();
  while ((millis() - startTime) < timeout) {
    while (sim800l.available()) {
      response += (char)sim800l.read();
    }
  }
  return response;
}

// SMS Sending Function
void sendSMS(String message) {
  sendATCommand("AT+CMGF=1", 500);
  sim800l.print("AT+CMGS=\"");
  sim800l.print(PHONE_NUMBER);
  sim800l.println("\"");
  delay(50);
  sim800l.print(message);
  delay(50);
```

CODES

```
  sim800l.write(26);  // Ctrl+Z to send
  Serial.println("📩 SMS sent: " + message);
}

// Initialize EEPROM
void initEEPROM() {
  EEPROM.begin(EEPROM_SIZE);

  // Read tag count from EEPROM
  tagCount = EEPROM.read(0);
  if (tagCount > MAX_TAGS) tagCount = 0;

  // Read tag data from EEPROM
  for (int i = 0; i < tagCount; i++) {
    int address = 1 + (i * TAG_DATA_SIZE);
    String tagData;
    for (int j = 0; j < TAG_DATA_SIZE; j++) {
      char c = EEPROM.read(address + j);
      if (c == '\0') break;
      tagData += c;
    }

    // Parse tag data (format: id|name|weight|lastScanTime)
    int firstPipe = tagData.indexOf('|');
    int secondPipe = tagData.indexOf('|', firstPipe + 1);
    int thirdPipe = tagData.indexOf('|', secondPipe + 1);

    if (firstPipe != -1 && secondPipe != -1 && thirdPipe != -1) {
      tagDatabase[i].tagId = tagData.substring(0, firstPipe);
      tagDatabase[i].tagName = tagData.substring(firstPipe + 1, secondPipe);
      tagDatabase[i].lastWeight = tagData.substring(secondPipe + 1, thirdPipe).toFloat();
      tagDatabase[i].lastScanTime = tagData.substring(thirdPipe + 1);
    }
  }
  Serial.println("📂 Loaded " + String(tagCount) + " tags from EEPROM");
}

// Save tag to EEPROM
void saveTagToEEPROM(const String &tagId, const String &tagName, float weight, const String &scanTime) {
  // Check if tag already exists
  int existingIndex = -1;
  for (int i = 0; i < tagCount; i++) {
    if (tagDatabase[i].tagId == tagId) {
      existingIndex = i;
      break;
    }
  }

  if (existingIndex == -1) {
    // Add new tag
    if (tagCount >= MAX_TAGS) {
      Serial.println("❌ Tag database full!");
      return;
    }
    existingIndex = tagCount;
    tagCount++;
  }

  // Update tag data
  tagDatabase[existingIndex].tagId = tagId;
  tagDatabase[existingIndex].tagName = tagName;
  tagDatabase[existingIndex].lastWeight = weight;
  tagDatabase[existingIndex].lastScanTime = scanTime;

  // Prepare data string (format: id|name|weight|lastScanTime)
  String tagData = tagId + "|" + tagName + "|" + String(weight, 2) + "|" + scanTime;

  // Write to EEPROM
  int address = 1 + (existingIndex * TAG_DATA_SIZE);
  for (int i = 0; i < tagData.length() && i < TAG_DATA_SIZE; i++) {
    EEPROM.write(address + i, tagData[i]);
  }
  // Null terminate
  if (tagData.length() < TAG_DATA_SIZE) {
    EEPROM.write(address + tagData.length(), '\0');
  }

  // Update tag count
  EEPROM.write(0, tagCount);
  EEPROM.commit();

  Serial.println("💾 Saved tag to EEPROM: " + tagId + " (" + tagName + ")");
}

// Delete tag from EEPROM
void deleteTagFromEEPROM(const String &tagId) {
  int indexToDelete = -1;

  // Find the tag
  for (int i = 0; i < tagCount; i++) {
    if (tagDatabase[i].tagId == tagId) {
      indexToDelete = i;
      break;
    }
  }

  if (indexToDelete == -1) return;

  // Shift all subsequent tags up
  for (int i = indexToDelete; i < tagCount - 1; i++) {
    tagDatabase[i] = tagDatabase[i + 1];

    // Update EEPROM
    String tagData = tagDatabase[i].tagId + "|" + tagDatabase[i].tagName + "|" +
                     String(tagDatabase[i].lastWeight, 2) + "|" + tagDatabase[i].lastScanTime;

    int address = 1 + (i * TAG_DATA_SIZE);
    for (int j = 0; j < tagData.length() && j < TAG_DATA_SIZE; j++) {
      EEPROM.write(address + j, tagData[j]);
    }
    if (tagData.length() < TAG_DATA_SIZE) {
      EEPROM.write(address + tagData.length(), '\0');
    }
  }
```

CODES

```
  // Clear the last tag
  int lastAddress = 1 + ((tagCount - 1) * TAG_DATA_SIZE);
  for (int i = 0; i < TAG_DATA_SIZE; i++) {
    EEPROM.write(lastAddress + i, 0);
  }

  tagCount--;
  EEPROM.write(0, tagCount);
  EEPROM.commit();

  Serial.println("🗑️ Deleted tag from EEPROM: " + tagId);
}

// Web Server Handlers
void handleRoot() {
  server.send(200, "text/html", htmlPage);
}

void handleManualFeed() {
  Serial.println("Manual feed activated via web");
  myservo.write(180);
  delay(1000);
  myservo.write(0);
  server.send(200, "text/plain", "Manual feed activated");
}

void handleWeighChicken() {
  Serial.println("Weighing chicken via web");
  // In a real implementation, you would trigger the scale reading here
  server.send(200, "text/plain", "Weighing chicken");
}

void handleTogglePump() {
  bool currentState = digitalRead(RELAY_PIN);
  digitalWrite(RELAY_PIN, !currentState);
  String state = !currentState ? "ON" : "OFF";
  Serial.println("Pump toggled via web: " + state);
  server.send(200, "text/plain", "Pump turned " + state);
}

void handleSaveTag() {
  if (server.hasArg("plain")) {
    String message = server.arg("plain");
    StaticJsonDocument<200> doc;
    DeserializationError error = deserializeJson(doc, message);

    if (error) {
      server.send(400, "text/plain", "Invalid JSON");
      return;
    }

    String tagId = doc["tagId"];
    String tagName = doc["tagName"];
    float tagWeight = doc["weight"];

    if (tagId.isEmpty() || tagName.isEmpty()) {
      server.send(400, "text/plain", "Bad Request: Missing parameters");
      return;
    }

    // Get current time
    struct tm timeinfo;
    String scanTime = "--:--:--";
    if (getLocalTime(&timeinfo)) {
      char buf[20];
      strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", &timeinfo);
      scanTime = String(buf);
    }

    saveTagToEEPROM(tagId, tagName, tagWeight, scanTime);
    server.send(200, "text/plain", "Tag saved successfully");
  } else {
    server.send(400, "text/plain", "No data received");
  }
}

oid handleDeleteTag() {
 if (server.hasArg("plain")) {
   String message = server.arg("plain");
   StaticJsonDocument<200> doc;
   DeserializationError error = deserializeJson(doc, message);

   if (error) {
     server.send(400, "text/plain", "Invalid JSON");
     return;
   }

    String tagId = doc["tagId"];
    if (tagId.isEmpty()) {
      server.send(400, "text/plain", "Bad Request: Missing tagId");
      return;
    }

    deleteTagFromEEPROM(tagId);
    server.send(200, "text/plain", "Tag deleted successfully");
  } else {
    server.send(400, "text/plain", "No data received");
  }
}

void handleData() {
  struct tm timeinfo;
  String timeStr = "--:--:--";
  if (getLocalTime(&timeinfo)) {
    char buf[20];
    strftime(buf, sizeof(buf), "%H:%M:%S", &timeinfo);
    timeStr = String(buf);
  }

  // Check water level
  int waterValue = analogRead(WATER_SENSOR_PIN);
  bool waterLow = waterValue < 200;

  // Get current weight
  float currentWeight = scaleReady ? scale.get_units(10) : 0.0;
```

CODES

```
  // Prepare schedule data
  String scheduleJson = "[";
  for (int i = 0; i < numFeeds; i++) {
    if (i > 0) scheduleJson += ",";
    scheduleJson += "{";
    scheduleJson += "\"time\":\"" + String(feedHours[i]) + ":" + (feedMinutes[i] < 10 ? "0" : "") + String(feedMinutes[i]) + "\",";
    scheduleJson += "\"status\":\"" + String(timeinfo.tm_hour == feedHours[i] && timeinfo.tm_min == feedMinutes[i] ? "Feeding now!" : "Pending") + "\"";
    scheduleJson += "}";
  }
  scheduleJson += "]";

  // Prepare tag data
  String tagsJson = "[";
  for (int i = 0; i < tagCount; i++) {
    if (i > 0) tagsJson += ",";
    tagsJson += "{";
    tagsJson += "\"tagId\":\"" + tagDatabase[i].tagId + "\",";
    tagsJson += "\"tagName\":\"" + tagDatabase[i].tagName + "\",";
    tagsJson += "\"lastWeight\":" + String(tagDatabase[i].lastWeight, 2) + ",";
    tagsJson += "\"lastScanTime\":\"" + tagDatabase[i].lastScanTime + "\"";
    tagsJson += "}";
  }
  tagsJson += "]";

  // Send JSON response
  String jsonResponse = "{";
  jsonResponse += "\"time\":\"" + timeStr + "\",";
  jsonResponse += "\"wifiStatus\":\"" + String(WiFi.status() == WL_CONNECTED ? "Connected" : "Disconnected") + "\",";
  jsonResponse += "\"wifiConnected\":" + String(WiFi.status() == WL_CONNECTED ? "true" : "false") + ",";
  jsonResponse += "\"rfidStatus\":\"" + String(rfid.PCD_PerformSelfTest() ? "Ready" : "Offline") + "\",";
  jsonResponse += "\"rfidReady\":" + String(rfid.PCD_PerformSelfTest() ? "true" : "false") + ",";
  jsonResponse += "\"scaleStatus\":\"" + String(scaleReady ? "Ready" : "Offline") + "\",";
  jsonResponse += "\"scaleReady\":" + String(scaleReady ? "true" : "false") + ",";
  jsonResponse += "\"waterLevel\":\"" + String(waterLow ? "Low" : "OK") + "\",";
  jsonResponse += "\"waterLow\":" + String(waterLow ? "true" : "false") + ",";
  jsonResponse += "\"pumpStatus\":" + String(digitalRead(RELAY_PIN) ? "true" : "false") + ",";
  jsonResponse += "\"currentWeight\":" + String(currentWeight, 2) + ",";
  jsonResponse += "\"rfidTag\":\"" + lastRFIDTag + "\",";
  jsonResponse += "\"schedule\":" + scheduleJson + ",";
  jsonResponse += "\"tags\":" + tagsJson;
  jsonResponse += "}";

  server.send(200, "application/json", jsonResponse);
}

void setup() {
  Serial.begin(115200);
  sim800l.begin(9600, SERIAL_8N1, SIM800L_RX, SIM800L_TX);
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);
  rfid.PCD_Init();
  myservo.attach(SERVO_PIN);
  pinMode(WATER_SENSOR_PIN, INPUT);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);

  // Initialize EEPROM
  initEEPROM();

  // Connect to WiFi
  WiFi.begin(wifiSSID, wifiPassword);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected to WiFi");

  // Start AP mode
  WiFi.softAP(apSSID, apPassword);
  WiFi.softAPConfig(localIP, gateway, subnet);

  // Start NTP
  configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);

  // Start web server
  server.on("/", handleRoot);
  server.on("/manualfeed", HTTP_POST, handleManualFeed);
  server.on("/weighchicken", HTTP_POST, handleWeighChicken);
  server.on("/togglepump", HTTP_POST, handleTogglePump);
  server.on("/savetag", HTTP_POST, handleSaveTag);
  server.on("/deletetag", HTTP_POST, handleDeleteTag);
  server.on("/data", HTTP_GET, handleData);
  server.begin();
```

CODES

```
  // Display network info
  displayNetworkInfo();

  scale.begin(HX711_DOUT, HX711_SCK);
  if (scale.is_ready()) {
    scale.set_scale(calibration_factor);
    scaleReady = true;
  } else {
    Serial.println("⚠️ HX711 not detected!");
  }
}

void loop() {
  server.handleClient();

  // Check RFID
  if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
    String tagId = "";
    for (byte i = 0; i < rfid.uid.size; i++) {
      tagId += String(rfid.uid.uidByte[i] < 0x10 ? "0" : "");
      tagId += String(rfid.uid.uidByte[i], HEX);
    }
    tagId.toUpperCase();

    // Check if tag is in database
    bool tagFound = false;
    for (int i = 0; i < tagCount; i++) {
      if (tagDatabase[i].tagId == tagId) {
        tagFound = true;
        lastRFIDTag = tagDatabase[i].tagName;
        Serial.println("RFID Tag Found: " + tagId + " (" + lastRFIDTag + ")");
        break;
      }
    }

    if (!tagFound) {
      // Add new tag with default values
      String defaultName = "New Tag";
      float defaultWeight = 0.0;
      String scanTime = getCurrentTime();
      saveTagToEEPROM(tagId, defaultName, defaultWeight, scanTime);
      lastRFIDTag = defaultName;
      Serial.println("New RFID Tag Added: " + tagId + " (" + defaultName + ")");
    }

    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();
  }

  // Check water level
  bool currentWaterState = analogRead(WATER_SENSOR_PIN) < 200;
  if (currentWaterState != lastWaterState) {
    lastWaterState = currentWaterState;
    if (currentWaterState) {
      sendSMS("🚨 Water level is low!");
    }
  }

  // Check feeding schedule
  struct tm timeinfo;
  if (getLocalTime(&timeinfo)) {
    for (int i = 0; i < numFeeds; i++) {
      if (timeinfo.tm_hour == feedHours[i] && timeinfo.tm_min == feedMinutes[i] &&
        (lastFeedHour != feedHours[i] || lastFeedMinute != feedMinutes[i])) {
        Serial.println("Feeding time! " + String(feedHours[i]) + ":" + (feedMinutes[i] < 10 ? "0" : "") +
String(feedMinutes[i]));
        myservo.write(180);
        delay(1000);
        myservo.write(0);
        lastFeedHour = feedHours[i];
        lastFeedMinute = feedMinutes[i];
      }
    }
  }

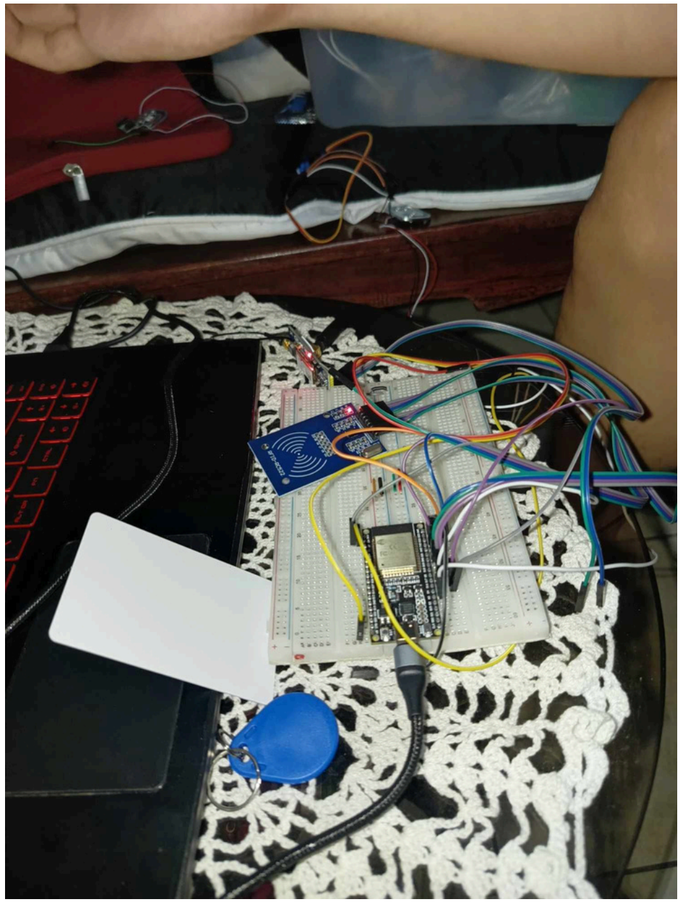
  // Print status to serial every 5 seconds
  if (millis() - lastSerialPrintTime >= serialPrintInterval) {
    lastSerialPrintTime = millis();
    Serial.println("Status:");
    Serial.println("- Last RFID Tag: " + lastRFIDTag);
    Serial.println("- Water Level: " + String(lastWaterState ? "Low" : "OK"));
  }
}

// Helper function to get the current time as a string
String getCurrentTime() {
  struct tm timeinfo;
  if (getLocalTime(&timeinfo)) {
    char buf[20];
    strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", &timeinfo);
    return String(buf);
  }
  return "--:--:--";
}
```

DOCUMENTATION







DOCUMENTATION

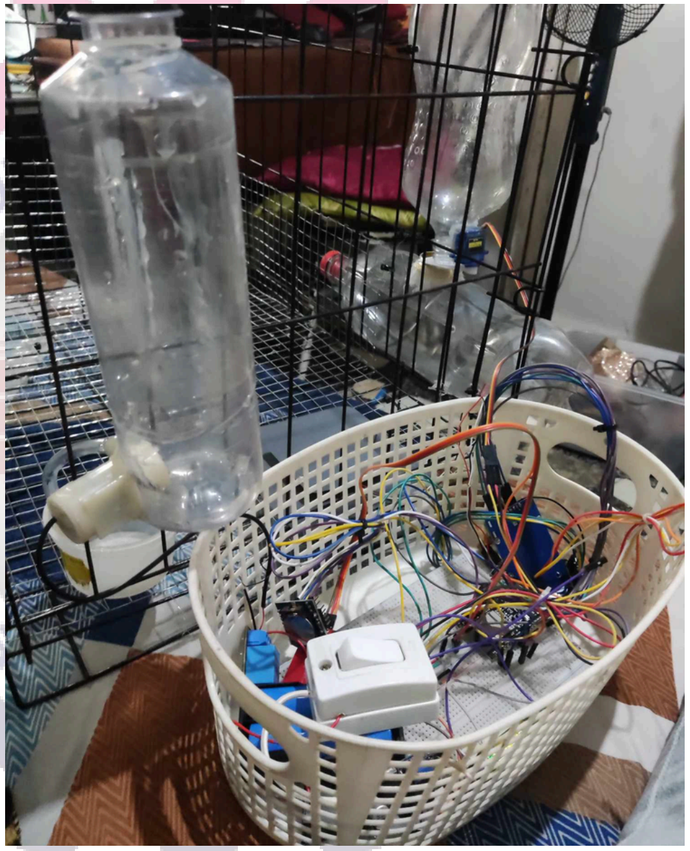






DOCUMENTATION







REFERENCES

- Automatic Feeding and Water Dispensing
  https://youtu.be/wQaPUCECcDw?si=Ag3uujY62nLRo3YW&t=354

- Weight Identification:
  https://youtu.be/rCjJjZ9gYs4?si=kmB0v7Kw2bxqokzr&t=200

- RFID scanner for radio tags
  https://www.rfidjournal.com/news/hen-behavior-tracked-with-rfid-at-university-of-georgia/202794/
  https://www.sciencedirect.com/science/article/pii/S003257912200706 4